



Rajshahi University of Engineering & Technology (RUET)

Department of Electrical and Computer Engineering (ECE)

Lab Report - 2

Course Code: ECE 2216

Course Title: Data Base Systems Sessional

Submitted by

Md. Yousuf
Student ID: 2310023
Series: 2023
Dept. of ECE, RUET

Submitted to

Mst. Nafia Islam Shish
Lecturer
Dept. of ECE, RUET

July 14, 2026

Contents

Theory	2
Table Creation and Data Insertion	3
Task Implementation	5
Discussion	11

Theory

Structured Query Language (SQL) is a standard language used to create, manage, and manipulate relational databases. A relational database organizes data into one or more tables, where each table consists of rows (records) and columns (fields/attributes).

The fundamental SQL operations used in this lab are broadly classified as follows:

- **DDL (Data Definition Language):** Commands such as `CREATE TABLE` are used to define the structure of a database table, including its columns and their data types (e.g., `VARCHAR`).
- **DML (Data Manipulation Language):** Commands such as `INSERT INTO` are used to add new records into a table, while `SELECT` is used to retrieve and query data based on specified conditions.
- **Filtering and Conditions:** The `WHERE` clause is used to filter records based on given conditions, often combined with logical operators such as `AND`, `OR`, and pattern-matching operators like `LIKE`.
- **Sorting and Grouping:** The `ORDER BY` clause arranges the result set in ascending (`ASC`) or descending (`DESC`) order, while `GROUP BY` combined with aggregate functions such as `COUNT()` is used to summarize data.
- **Joins and Set Operations:** The `INNER JOIN` clause combines rows from two or more tables based on a related column, while set operations such as `UNION` combine and de-duplicate the results of two queries. Subqueries with `NOT IN` help identify records present in one table but absent in another.

In this lab, two tables, `Scientist` and `Scientist_Conference`, were created and populated to practice these DDL, DML, and query operations in a hands-on manner using the Programiz Online SQL Editor (SQLite).

Table Creation and Data Insertion

Two tables were created for this lab: **Scientist**, which stores the core profile information of scientists, and **Scientist_Conference**, which stores the subset of scientists registered for a conference along with additional attendees.

Table 1: Scientist

The **Scientist** table was created with six attributes – **Scientist_ID**, **Scientist_Name**, **Specialization**, **Institution**, **Country**, and **Email** – all defined as **VARCHAR(50)**. Data for five scientists was then inserted using the **INSERT INTO ... VALUES** statement, with **Scientist_ID** values following the roll-number-based format (2310023–2310027).

The screenshot shows an online SQL editor interface. The 'Input' tab is active, displaying the following SQL code:

```
CREATE TABLE Scientist(  
  Scientist_ID varchar(50),  
  Scientist_Name varchar(50),  
  Specialization varchar(50),  
  Institution varchar(50),  
  Country varchar(50),  
  Email varchar(50)  
);  
  
INSERT INTO Scientist (Scientist_ID, Scientist_Name, Specialization, Institution, Country, Email)  
VALUES  
(  
'2310023', 'Jennifer Doudna', 'Human Genetics', 'University of California', 'USA', 'jdoudna@uc.edu'),  
(  
'2310024', 'Emmanuelle Charpentier', 'Molecular Biology', 'Max Planck Unit', 'Germany',  
'echarpentier@mpg.de'),  
(  
'2310025', 'Francis Collins', 'Human Genetics', 'NIH', 'USA', 'fcollins@nih.gov'),  
(  
'2310026', 'Craig Venter', 'Genomics', 'J. Craig Venter Institute', 'USA', 'cventer@jvvi.org'),  
(  
'2310027', 'Eric Lander', 'Computational Biology', 'Broad Institute', 'USA',  
'elander@broadinstitute.org');
```

The 'Output' tab shows the message: "SQL query successfully executed. However, the result set is empty."

On the right side, there is a section titled 'Available Tables' with a table listing two tables:

Table Name	Rows	Columns
Keyboard	400	1
Mousepad	250	2

Below this, there is a section titled 'Scientist' with a table showing the data inserted into the **Scientist** table:

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Figure 1: CREATE TABLE and INSERT INTO statements for the Scientist table

Table 2: Scientist_Conference

The **Scientist_Conference** table was created with three attributes – **Scientist_ID**, **Scientist_Name**, and **Specialization** – to record scientists attending a conference. It was populated using an **INSERT INTO ... VALUES** statement containing both scientists already present in the **Scientist** table and additional scientists exclusive to the conference.

These two tables together allow practicing both single-table queries and multi-table operations such as joins and set operations, as demonstrated in the following tasks.

Programiz Online SQL Editor [Interactive SQL Course](#)

> Input ⌂ ↺ ⋮ Run SQL > Available Tables

```
CREATE TABLE Scientist_Conference(  
  Scientist_ID varchar(50),  
  Scientist_Name varchar(50),  
  Specialization varchar(50)  
);  
  
INSERT INTO Scientist_Conference (Scientist_ID, Scientist_Name, Specialization)  
VALUES  
( '2310025', 'Francis Collins', 'Human Genetics'),  
( '2310026', 'Craig Venter', 'Genomics'),  
( '2310027', 'Eric Lander', 'Computational Biology'),  
( '2310028', 'Katalin Karikó', 'mRNA Technology'),  
( '2310029', 'Svante Pääbo', 'Evolutionary Genetics');
```

Output

SQL query successfully executed. However, the result set is empty.

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Figure 2: CREATE TABLE and INSERT INTO statements for the Scientist_Conference table

Task Implementation

Task 1: Scientists with Specialization “Genomics”

This query filters the `Scientist` table to display all records where the `Specialization` column matches the value 'Genomics' exactly, using a simple `WHERE` condition.

```
1 SELECT * FROM Scientist
2 WHERE Specialization = 'Genomics';
```

The screenshot shows the Programiz Online SQL Editor interface. The 'Input' tab contains the SQL query: `SELECT * FROM Scientist WHERE Specialization = 'Genomics';`. The 'Run SQL' button is visible. The 'Output' tab displays the results of the query in a table format. The 'Available Tables' section on the right lists the tables in the database: `Scientist` and `Scientist_Conference`.

Scientist_ID	Scientist_Name	Specialization	Institution	Country	Email
2310026	Craig Venter	Genomics	J. Craig Venter Institute	USA	cventer@jvvi.org

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Figure 3: Output of Task 1

Task 2: USA Scientists with “Institute” in Institution Name

This query combines two conditions using the `AND` operator: it checks that the `Country` is 'USA' and that the `Institution` field contains the substring "Institute", using the `LIKE` operator with wildcard characters (%).

```
1 SELECT * FROM Scientist
2 WHERE Country = 'USA' AND Institution LIKE '%Institute%';
```

Programiz Online SQL Editor

Interactive SQL Course

Input

```
SELECT * FROM Scientist
WHERE Country = 'USA' AND Institution LIKE '%Institute%';
```

Run SQL

Available Tables

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Output

Scientist_ID	Scientist_Name	Specialization	Institution	Country	Email
2310026	Craig Venter	Genomics	J. Craig Venter Institute	USA	cventer@jvvi.org
2310027	Eric Lander	Computational Biology	Broad Institute	USA	elander@broadinstitute.org

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Figure 4: Output of Task 2

Task 3: Names and Emails of Scientists from Germany

This query uses column selection to retrieve only the `Scientist_Name` and `Email` fields, filtered where `Country` equals `'Germany'`.

```
1 SELECT Scientist_Name, Email FROM Scientist
2 WHERE Country = 'Germany';
```

Programiz Online SQL Editor

Interactive SQL Course

Input

```
SELECT Scientist_Name, Email FROM Scientist
WHERE Country = 'Germany';
```

Run SQL

Available Tables

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Output

Scientist_Name	Email
Emmanuelle Charpentier	echarpentier@mpg.de

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Figure 5: Output of Task 3

Task 4: Names and Specializations Sorted Alphabetically

This query retrieves the `Scientist_Name` and `Specialization` columns and arranges the result set in ascending alphabetical order using the `ORDER BY` clause.

```

1 SELECT Scientist_Name, Specialization FROM Scientist
2 ORDER BY Scientist_Name ASC;

```

Programiz Online SQL Editor [Interactive SQL Course](#)

Input [Run SQL](#) Available Tables

```
SELECT Scientist_Name, Specialization FROM Scientist
ORDER BY Scientist_Name ASC;
```

Output

Scientist_Name	Specialization
Craig Venter	Genomics
Emmanuelle Charpentier	Molecular Biology
Eric Lander	Computational Biology
Francis Collins	Human Genetics
Jennifer Doudna	Human Genetics

Available Tables

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Figure 6: Output of Task 4

Task 5: Count of Scientists per Country

This query uses the **GROUP BY** clause along with the aggregate function **COUNT(*)** to determine how many scientists belong to each country.

```

1 SELECT Country, COUNT(*) AS Total_Scientists FROM Scientist
2 GROUP BY Country;

```

Programiz Online SQL Editor [Interactive SQL Course](#)

Input [Run SQL](#) Available Tables

```
SELECT Country, COUNT(*) AS Total_Scientists FROM Scientist
GROUP BY Country;
```

Output

Country	Total_Scientists
Germany	1
USA	4

Available Tables

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Figure 7: Output of Task 5

Task 6: Scientists with Email Addresses Ending in “.org”

This query uses the `LIKE` operator with a leading wildcard (`%.org`) to match email addresses whose domain ends with `.org`.

```
1 SELECT * FROM Scientist
2 WHERE Email LIKE '%.org';
```

The screenshot shows an Online SQL Editor interface. The input field contains the SQL query: `SELECT * FROM Scientist WHERE Email LIKE '%.org';`. The output displays two tables: `Scientist` and `Scientist_Conference`.

Scientist Table:

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_Conference Table:

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Output Table (Full Results):

Scientist_ID	Scientist_Name	Specialization	Institution	Country	Email
2310026	Craig Venter	Genomics	J. Craig Venter Institute	USA	cventer@jvvi.org
2310027	Eric Lander	Computational Biology	Broad Institute	USA	elander@broadinstitute.org

Figure 8: Output of Task 6

Task 7: Scientists Present in Both Tables (INNER JOIN)

This query uses an `INNER JOIN` between the `Scientist` and `Scientist_Conference` tables on the common `Scientist_ID` column, returning only the names and specializations of scientists who appear in both tables.

```
1 SELECT S.Scientist_Name, S.Specialization
2 FROM Scientist S
3 INNER JOIN Scientist_Conference SC
4 ON S.Scientist_ID = SC.Scientist_ID;
```

Task 8: Unique Scientists from Both Tables (UNION)

This query uses the `UNION` set operator to combine the results of the `Scientist` and `Scientist_Conference` tables while automatically removing fully duplicate rows.

```
1 SELECT Scientist_ID, Scientist_Name, Specialization FROM
   Scientist
2 UNION
```

Programiz Online SQL Editor [Interactive SQL Course](#)

Input

```
SELECT S.Scientist_Name, S.Specialization
FROM Scientist S
INNER JOIN Scientist_Conference SC
ON S.Scientist_ID = SC.Scientist_ID;
```

Run SQL

Available Tables

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Output

Scientist_Name	Specialization
Francis Collins	Human Genetics
Craig Venter	Genomics
Eric Lander	Computational Biology

Figure 9: Output of Task 7

```
3 SELECT Scientist_ID, Scientist_Name, Specialization FROM
   Scientist_Conference;
```

Programiz Online SQL Editor [Interactive SQL Course](#)

Input

```
SELECT Scientist_ID, Scientist_Name, Specialization FROM Scientist
UNION
SELECT Scientist_ID, Scientist_Name, Specialization FROM Scientist_Conference;
```

Run SQL

Available Tables

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Output

Scientist_ID	Scientist_Name	Specialization
2310023	Jennifer Doudna	Human Genetics
2310024	Emmanuelle Charpentier	Molecular Biology
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Shippings

Figure 10: Output of Task 8

Task 9: Scientists in Scientist Table but Not in Scientist_Conference

This query uses a subquery with the NOT IN operator to find scientists whose Scientist_ID does not appear anywhere in the Scientist_Conference table.

```
1 SELECT * FROM Scientist
2 WHERE Scientist_ID NOT IN (SELECT Scientist_ID FROM
   Scientist_Conference);
```

Programiz Online SQL Editor [Interactive SQL Course](#)

Input

```
SELECT * FROM Scientist
WHERE Scientist_ID NOT IN (SELECT Scientist_ID FROM Scientist_Conference);
```

Run SQL

Available Tables

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Output

Scientist_ID	Scientist_Name	Specialization	Institution	Country	Email
2310023	Jennifer Doudna	Human Genetics	University of California	USA	jdoudna@uc.edu
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit	Germany	echarpentier@mpg.de

Figure 11: Output of Task 9

Task 10: Scientists from USA with Specialization “Human Genetics”

This query combines two exact-match conditions using the AND operator to filter scientists who are both from 'USA' and specialize in 'Human Genetics'.

```
1 SELECT * FROM Scientist
2 WHERE Country = 'USA' AND Specialization = 'Human Genetics';
```

Programiz Online SQL Editor [Interactive SQL Course](#)

Input

```
SELECT * FROM Scientist
WHERE Country = 'USA' AND Specialization = 'Human Genetics';
```

Run SQL

Available Tables

Scientist

Scientist_ID	Scientist_Name	Specialization	Institution
2310023	Jennifer Doudna	Human Genetics	University of California
2310024	Emmanuelle Charpentier	Molecular Biology	Max Planck Unit
2310025	Francis Collins	Human Genetics	NIH
2310026	Craig Venter	Genomics	J. Craig Venter Institute
2310027	Eric Lander	Computational Biology	Broad Institute

Scientist_Conference

Scientist_ID	Scientist_Name	Specialization
2310025	Francis Collins	Human Genetics
2310026	Craig Venter	Genomics
2310027	Eric Lander	Computational Biology
2310028	Katalin Karikó	mRNA Technology
2310029	Svante Pääbo	Evolutionary Genetics

Output

Scientist_ID	Scientist_Name	Specialization	Institution	Country	Email
2310023	Jennifer Doudna	Human Genetics	University of California	USA	jdoudna@uc.edu
2310025	Francis Collins	Human Genetics	NIH	USA	fcollins@nih.gov

Figure 12: Output of Task 10

Discussion

In this lab, two relational tables, `Scientist` and `Scientist_Conference`, were created and populated using the `CREATE TABLE` and `INSERT INTO` statements in the Programiz Online SQL Editor. Through a series of ten tasks, various SQL querying techniques were practiced, including:

- Filtering records using `WHERE` with exact matches, the `AND` logical operator, and pattern matching with `LIKE`.
- Selecting specific columns instead of the entire table.
- Sorting results using `ORDER BY`.
- Aggregating and summarizing data using `GROUP BY` with `COUNT()`.
- Combining data from multiple tables using `INNER JOIN`.
- Merging and de-duplicating result sets from multiple tables using `UNION`.
- Identifying records unique to one table using a subquery with `NOT IN`.

Through these exercises, a clearer understanding was developed of how relational databases store, filter, sort, and combine data, and how SQL provides a compact and powerful way to answer complex questions about structured data spanning multiple tables. This lab reinforced the practical importance of consistent primary key usage (`Scientist_ID`) across related tables, since join and set operations depend directly on matching key values between tables.